

Сервис прогнозирования отказов датчиков инженерных систем ЖКХ

Сопроводительная документация к конкурсному прототипу

Департамент ЖКХ города Москвы · АО «Москоллектор» — направление «Отказ датчика»

Версия документа: 22 сентября 2026 года

1. Общее описание решения

Веб-сервис прогнозирования отказов датчиков инженерных систем (насос/вентилятор, дым/газ) с диспетчерским интерфейсом, объяснимой ML-моделью и формированием заявок на превентивное обслуживание по решению диспетчера. Прототип развёрнут и доступен по адресу `jkh.devcosmos.ru`.

Ключевые задачи, которые решает сервис:

- прогнозирование отказа датчика на горизонте 24 часа с указанием вероятности и объяснением;
- приоритизация внимания диспетчера — короткий список рисков вместо потока сырых событий;
- фиксация решений диспетчера (наблюдать / направить на проверку / отклонить / уточнить) с журналом аудита;
- формирование заявки на обслуживание по явному решению диспетчера (автосоздание по правилу отключено — решение принимает человек).
- аналитика и отчётность по историческим инцидентам, сезонности и заявкам на обслуживание (экспорт XLSX/PDF), реестр версий моделей с метриками.

Документ закрывает требования раздела 14 ЖКХ.md: используемые методы обработки данных, условия и ограничения решения, инструкции по сборке/установке, функциональную и компонентную архитектуру.

2. Функциональная и компонентная архитектура

2.1. Компоненты системы

Компонент	Технологии	Роль
backend	FastAPI, SQLAlchemy, Alembic (Python 3.12)	REST API, авторизация (JWT), бизнес-логика, миграции схемы
worker	Тот же backend-код + CatBoost, scikit-learn, pandas	Фоновый процесс: проигрывает поток исторических событий, считает признаки, вызывает модель, создаёт риск-кейсы (заявку — по решению диспетчера)
frontend	React 18 + TypeScript, Vite, Tailwind CSS	Веб-интерфейс диспетчера: дашборд, риски, журнал, заявки, реестр объектов/каналов
db	PostgreSQL 16	Единственное хранилище состояния — оперативные данные, история прогнозов и решений
caddy (только прод)	Caddy 2	Обратный прокси, автоматический TLS-сертификат (Let's Encrypt)

2.2. Поток данных

- 1) Исторический импорт: CSV-архивы → `scripts/import_analysis_to_db.py` → PostgreSQL (каналы, объекты, эпизоды неисправностей, версии моделей, риск-кейсы, прогнозы).
- 2) Живой инференс: `worker` проигрывает тот же поток событий через тот же путь обработки, что будет использовать реальный адаптер источника (СМБУ) — на каждом тике считывает поведенческие признаки по каналу, вызывает обученную модель (`CatBoost.predict_proba`), при превышении порога создаёт риск-кейс с SHAP-объяснением; заявка на обслуживание создаётся только явным решением диспетчера на карточке риска, автосоздания по правилу нет.
- 3) Frontend обращается только к REST API backend'a — не имеет прямого доступа к БД. Все данные на экране — сохранённые записи, а не значения, нарисованные на клиенте.

2.3. Модель данных (основные сущности)

Object (объект, иерархия), Channel (канал датчика, привязан к объекту и, где сопоставлено, к физическому Device), IncidentEpisode (очищенный эпизод неисправности), ModelVersion, Prediction (журнал прогнозов с вероятностью и SHAP), RiskCase (открытый риск, статус, приоритет), Decision (решение диспетчера), MaintenanceRequest (заявка со сменой статусов), User/UserObjectAccess (роли и доступ по объектам), AuditLog (журнал действий).

3. Используемые методы обработки данных

3.1. Подготовка данных

- Аудит 8 годовых архивов (2019–2026, ~313 млн событий, ~16 ГБ), манифест с SHA-256, полный профиль по годам.
- Построение очищенных эпизодов отказа (насос, вентилятор, дым, газ) двухслойным декойзингом: фильтр дневного флаппинга (>10 переходов/сутки на канал) + `debounce` устойчивости ≥1 час.
- Политика разметки без доступа к эксперту зафиксирована отдельно (`docs/label-policy.md`): целевая метка — «переход в устойчивое состояние “Неисправен”» в ближайшие 24 часа.
- Соответствие канал → физическое устройство для насос/вентилятор построено эвристически: идентификатор устройства (номер + пикет) парсится из свободнотекстового названия датчика, покрытие 87.8% каналов (469/534).

3.2. Признаки и модель

- Витрина признаков на часовой сетке: окна 1ч/24ч/7сут, свежесть последнего события, соседи по расположению.
- Два независимых `CatBoostClassifier` (насос/вентилятор и дым/газ) с временным `train/validation/test` и разрывом 24 часа на границах — без утечки будущего в обучение.
- Базовая линия для сравнения — простая эвристика, ROC-AUC которой заведомо хуже `CatBoost` (0.54 против 0.71 на насос/вентилятор).
- Объяснение каждого прогноза — SHAP-значения из самого `CatBoost` (`get_feature_importance`, `type=ShapValues`), а не отдельная библиотека.
- Поверх SHAP — необязательный слой удобства: краткое резюме прогноза на естественном языке от LLM (через прокси-провайдера), пересказывающее те же SHAP-факторы словами для диспетчера; не подменяет и не заменяет объяснение модели.
- Независимый сигнал аномальности — необученный `IsolationForest` на тех же поведенческих признаках, способен отметить поведение вне размеченных сценариев отказа.
- Дополнительный сигнал — тренд деградации канала: сравнение частоты «чистых» (не флаппинг) эпизодов за недавний и базовый 90-дневные периоды (`worsening/stable/improving`).

3.3. Честная оценка качества

Целевые показатели Precision > 0.7 и Recall > 0.5 не достигнуты ни для насос/вентилятор (ROC-AUC 0.71), ни для дым/газ (ROC-AUC 0.63) — притом что оба заметно лучше случайного угадывания (0.5) и базовой линии. На эпизодном уровне лучший проверенный порог ловит около 15% реальных эпизодов при ~37 ложных сериях предупреждений на каждую верную. Три проверенные гипотезы улучшения признаков (соседние устройства в отказе, дневная гранулярность, числовые показания газовых каналов) не дали качественного прорыва. Полный разбор с методикой — docs/analysis/model_report_насос_вентилятор.md и model_report_дым_газ.md. Организаторы подтвердили (см. Приложение А), что целевые метрики плановые, а не жёсткие требования, и их можно снижать при обоснованном разборе причин — рабочая точка порога выбрана по эпизодной оценке, а не по формальному 0.7/0.5.

4. Условия и ограничения решения

Полная таблица — docs/Матрица_соответствия_T3.md. Ниже сведены пункты, отклонённые от базового T3, с обоснованием.

Пункт T3	Статус	Обоснование
Интерактивная GPS-карта (раздел 10)	Заменено	Организаторы подтвердили: реальных координат объектов не будет никогда (только текущие, теряются при демонтаже датчика) — иерархическая схема объектов с цветовой индикацией риска рекомендована ими же вместо карты.
Сценарный анализ пожар/проникновение/подтопление (раздел 6)	Не реализовано	Вне объёма двухнедельного MVP; направления оцениваются организаторами независимо, жёсткого приоритета нет. Трек дым/газ частично касается пожарного риска теми же датчиками, но не заявлен как отдельное направление.
Интеграция с СМБУ, журналами ОДС, внешним реестром оборудования (разделы 6, 7, 10)	Заменено имитацией	Реального доступа нет и не будет (подтверждено организаторами) — заменено воспроизведением исторических событий через тот же путь обработки, что будет использовать реальный адаптер источника.
LDAP/AD (раздел 11)	Не реализовано	Тестового контура LDAP/AD на время мероприятия нет; локальная JWT-авторизация с ролями и матрицей доступа по объектам — принятое временное решение для MVP.
RBAC по подразделениям (раздел 11)	Частично	Реализованы роли (admin/dispatcher) и доступ по объектам (UserObjectAccess); полная матрица по организационным подразделениям — задел для будущего пилота.
Push/WebSocket-уведомления (раздел 10)	Заменено периодически опросом	Организаторы подтвердили: push/email/SMS/эскалации желательны, но не обязательны для MVP — реализовано клиентское уведомление (тост + счётчик в меню) на периодическом опросе раз в 30 секунд.
Синхронизация с внешней базой оборудования (раздел 10)	Не реализовано	Внешний реестр оборудования не передан и не будет передан организаторами. Внутреннее соответствие канал → устройство построено эвристически из собственных данных (см. раздел 3.1).

5. Инструкция по сборке, установке и запуску

5.1. Требования к серверу

- Linux (Ubuntu 22.04/24.04 LTS), Docker Engine + Docker Compose plugin.
- Рекомендуется 4 vCPU / 8 ГБ RAM / 100 ГБ SSD (минимум — 2 vCPU / 4 ГБ / 50 ГБ).
- Публичный IP и домен (для TLS через Let's Encrypt) — только для прод-развёртывания.

5.2. Локальный запуск (разработка/проверка)

```
git clone <репозиторий> jkh && cd jkh
cp .env.example .env # заполнить пароли/секреты – не коммитить
docker compose -f docker-compose.yml -f docker-compose.dev.yml up -d --build
docker compose exec backend alembic upgrade head
```

Frontend будет доступен на <http://localhost:5174>, backend API — на <http://localhost:8000/api> (Swagger UI: /docs).

5.3. Прод-развёртывание

```
export DOMAIN=<ваш-домен> # должен указывать А-записью на IP сервера
docker compose -f docker-compose.yml -f deploy/docker-compose.prod.yml up -d --build
docker compose exec backend alembic upgrade head
```

Caddy автоматически получает и обновляет TLS-сертификат Let's Encrypt по домену DOMAIN.

5.4. Тестирование и проверка качества

- Backend test suite: 90 тестов (pytest), требует отдельный тестовый Postgres — см. backend/README.md.
- Нагрузочная проверка 20 параллельных пользователей — 0 ошибок на всех ключевых эндпоинтах (ApacheBench, см. docs/Статус.md, запись 21 сентября).
- Backup/restore: deploy/backup.sh / deploy/restore.sh (pg_dump -Fc / pg_restore --clean), время в разы меньше норматива ≤4 часа; реально сработало на проде при инциденте с потерей данных пользователей.

```
docker run -d --name jkh_test_db -e POSTGRES_USER=jkh -e POSTGRES_PASSWORD=jkh \
-e POSTGRES_DB=jkh_test -p 55432:5432 postgres:16
cd backend && python3 -m pytest tests/ -v
```

6. Реестр библиотек и компонентов

Полный перечень использованных библиотек, их версий, лицензий и назначения — docs/Лицензии_и_зависимости.md. Кратко по слоям: backend — FastAPI/SQLAlchemy/Alembic/Pydantic (все MIT), worker дополнительно — CatBoost/scikit-learn/pandas/pyarrow (Apache-2.0/BSD-3), frontend — React/TypeScript/Vite/Tailwind (MIT/Apache-2.0), инфраструктура — PostgreSQL 16, nginx, Caddy 2. Все лицензии — permissive open source, без ограничений на использование в конкурсном прототипе. Отдельного пакета для SHAP не используется — расчёт встроен в CatBoost.

7. Нормативное соответствие

Исходный код открыт, не откомпилирован, не обфусцирован. Работа с персональными данными ограничена собственными учётными записями пользователей сервиса (логин, роль, журнал действий) —

пароли хранятся в виде bcrypt-хеша, передача данных — по TLS 1.2+. Исходные данные конкурса обезличены на стороне организаторов до передачи. Соответствует требованиям 149-ФЗ и 152-ФЗ в объеме, применимом к прототипу такого масштаба.